

네트워크 게임 프로그래밍

6팀 추진 계획서

20211800009 박경준

20211800010 박대원

20211800035 임찬진

목차

1. 애플리케이션 기획
2. HIGH LEVEL DESIGN
3. LOW LEVEL DESIGN
4. 역할 분담
5. 개발 환경
6. 개별 일정

애플리케이션 기획

제목 : Geometry Dash

제작자 : 박대원, 민정원

제작한 수업 : 윈도우 프로그래밍

● 게임 소개

플레이어가 장애물을 피해 끝인 지점까지 도달하는 것이 목표인 게임이다.

플레이어의 동작 버튼이 Space Bar 하나로 간단하며 게임 내부에서는 다양한 타일들이 존재합니다.

플레이어는 직접 맵을 제작할 수 있어 이용자의 흥미를 둘 수 있습니다.

● 기존에 있던 기능

싱글플레이 모드

● 새로 추가할 기능

1vs1vs1 모드

● 키 입력 방식

◎ 실제 게임 플레이에서 사용

- 일반 모드

Space Bar를 눌러 점프가 가능합니다.

2단 점프 블록과 충돌 시 타이밍에 맞게 Space Bar를 누르면 공중에서 점프가 가능합니다.

- 비행 모드

Space Bar를 누르고있으면 위로 올라갑니다.

Space Bar를 떼면 내려갑니다.

- 지그재그 모드

Space Bar를 누르고있으면 위로 올라갑니다.

Space Bar를 떼면 내려갑니다.

◎ 모든 상황에서 사용

각 단계에서는 ESC를 눌러 지도나 메인화면으로 나갈 수 있습니다.

● 각 씬 설명

◎ 메인 메뉴 씬

게임의 대표 메인메뉴 입니다. 구성은 제목과 플레이 버튼입니다.

◎ 레벨 선택 씬

어떤 게임 모드를 플레이 할지에 대한 선택 버튼입니다. 2개의 모드가 있으며 각각

1. 기본 게임 씬
2. 1vs1vs1 씬

위 2가지 모드를 플레이 할 수 있는 버튼을 제작해줄 예정입니다.

◎ 1vs1vs1 씬

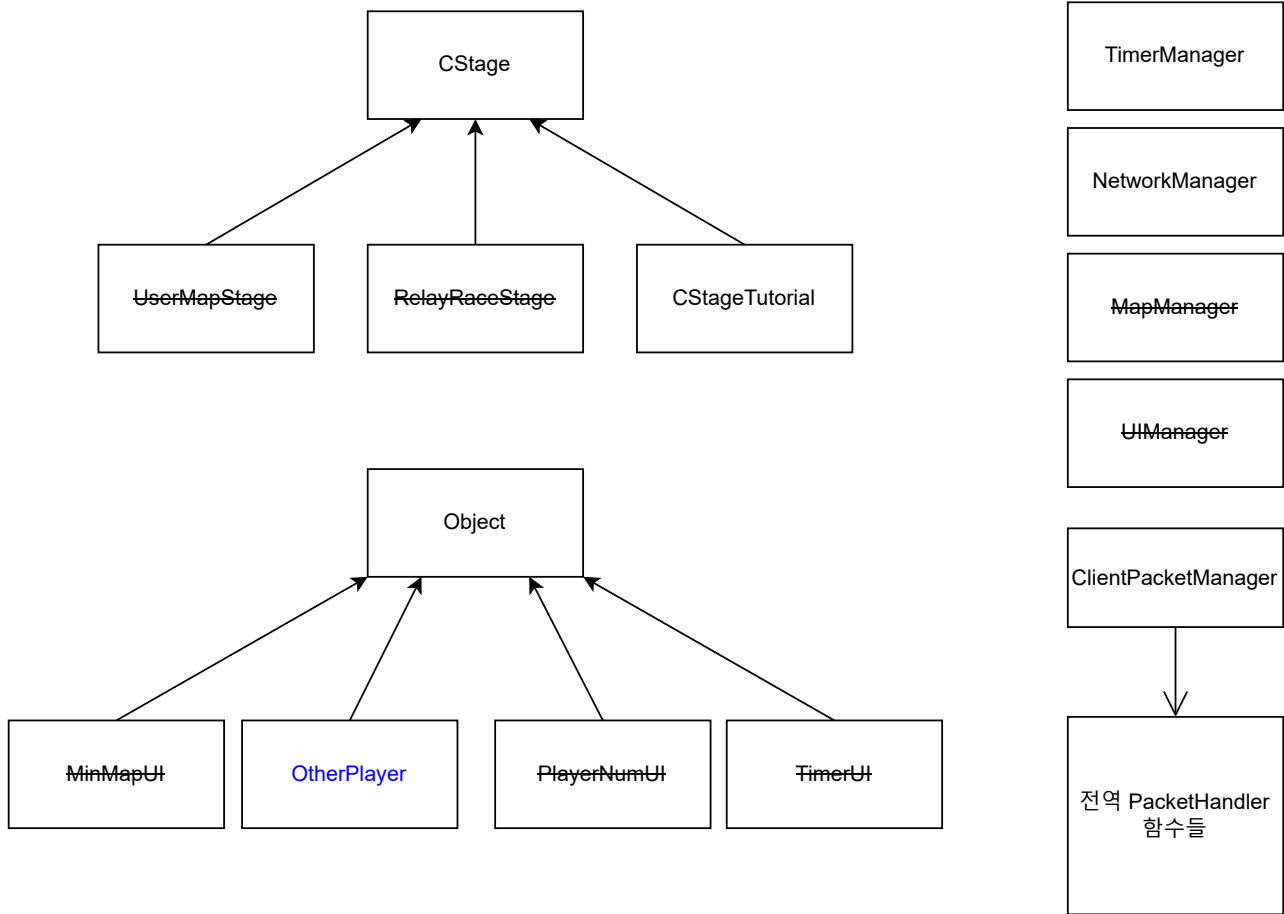
3명의 플레이어가 제작자가 미리 제작해놓은 맵 중 랜덤한 맵을 하나 플레이하며 가장 먼저 죽지않고 결승선에 도착한 플레이어가 우승하게되는 게임입니다.

화면에는 자신 뿐만 아니라 모든 사람들의 플레이어가 함께 달리도록 제작할 예정입니다.

- 화면 구성

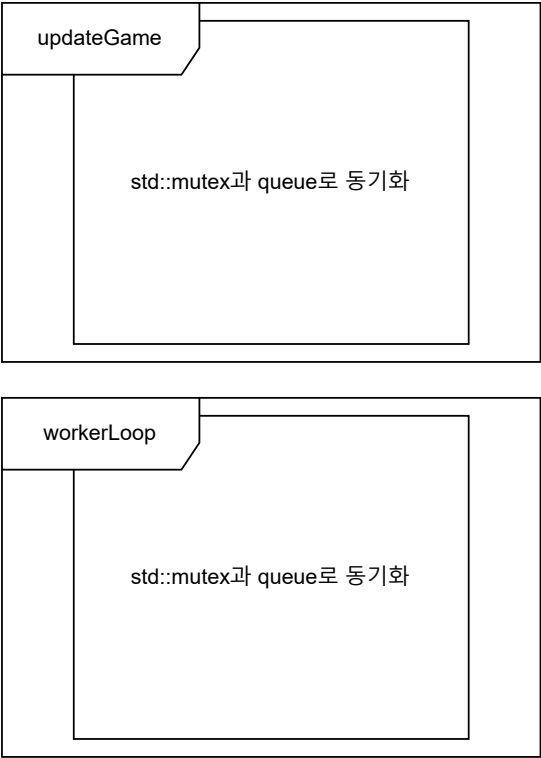
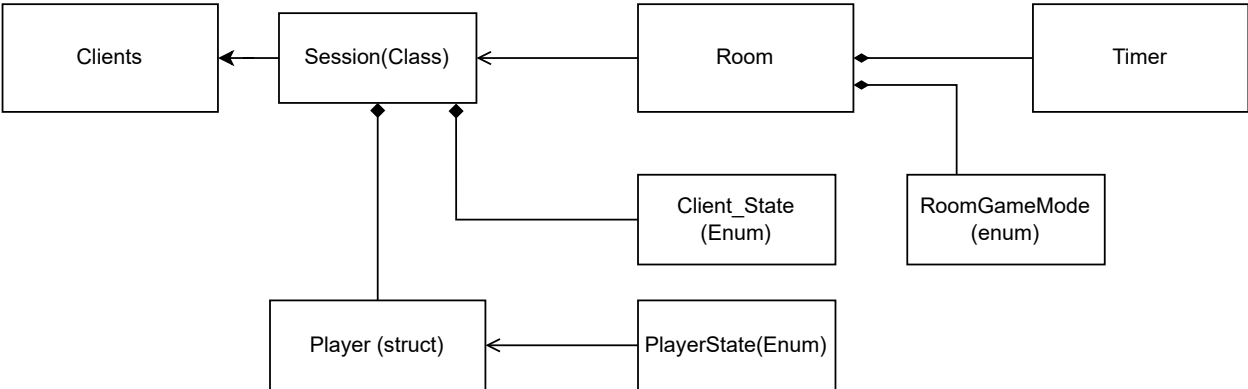
우측 상단에는 시간, 중앙 상단에는 미니맵을 띄워 해당 맵의 길이에 대한 모든 플레이어들의 진행 상황을 확인할 수 있도록 제작할 예정입니다.
그리고 좌측 상단에는 플레이어의 번호를 띄워줄 예정입니다.

Client High Level Design



Server

Server High Level Design



Client Low Level Design

Stage

```
Stage();
virtual ~Stage();

virtual void Initailizer() = 0;

virtual void Update(float fTime) = 0;

virtual void RestartStage() {}

virtual void LateUpdate(float fTime) = 0;

virtual void Render(HDC mDC) = 0;

virtual void Free() = 0;
```

CStageTutorial : CStage

```
CStageTutorial();
virtual ~CStageTutorial();

virtual void Initailizer() override;

virtual void RestartStage() override;

virtual void Update(float fTime) override;

virtual void LateUpdate(float fTime) override;

virtual void Render(HDC mDC) override;

virtual void Free() override;
```

TimerManager : Singleton

```
TimerManager();
virtual ~TimerManager();

void init();
void Reset();
void Tick();

float GetDeltaTime() const;
float GetFPS() const;

float SetTotalTime(float time);
float GetTotalTime() const;
```

Object

```
Object();
virtual ~Object();

virtual void Initailizer() = 0;

virtual void Update(float fTime) = 0;

virtual void LateUpdate(float fTime) = 0;

virtual void Render(HDC mDC) = 0;

virtual void Free() = 0;

SetPosRect();
GetPosRect();

SetCenterPos();
GetCenterPos();

ResetPosRect();

SetRidius();

GetRidius();

SetDead();

변수
bool m_bDead{};
RECT m_POSRECT{};
POINT m_CenterPos{};
float m_fRidius{};
```

OtherPlayer : Object

```
OtherPlayer();
virtual ~OtherPlayer();

virtual void Initailizer() override;

virtual void Update(float fTime) override;

virtual void LateUpdate(float fTime) override;

virtual void Render(HDC mDC) override;

virtual void Free() override;
```

ObjManager : Object

```
OtherPlayer();
virtual ~OtherPlayer();

virtual void Initailizer() override;

virtual void Update(float fTime) override;

virtual void LateUpdate(float fTime) override;

virtual void Render(HDC mDC) override;

virtual void Free() override;

void SetMyPlayer(CPlayer* _player);

void SetMyPlayerID(int id);

int GetMyPlayerID() const;

COtherPlayer* FindOtherPlayer(int id);

void SetMyPlayer(CPlayer* _player);

void SetMyPlayerID(int id);

int GetMyPlayerID() const;

COtherPlayer* FindOtherPlayer(int id);

void AddOtherPlayer(int id, float x, float y);

void RemoveOtherPlayer(int id);
```

NetworkManager : Singleton

```
NetworkManager();
virtual ~NetworkManager();

bool initialize_Client();

void sendPacket(char* buffer, int size);

void trySendBuffer();

void processPacket();

void updatePacket();

void shutdown();
```

전역 핸들러 함수

```
Handle_AllPlayerMovePacket_s2c

AllPlayer_MovePacket_s2c

Handle_LoginAccept_s2c

Handle_RoomWait_s2c

Handle_RoomStart_s2c

SpawnOtherPlayerPacket_s2c

DespawnOtherPlayerPaccke_s2c
```

<u>MapManager</u>
GetInstance(); DestoryInstance(); MapShow(); MapLoad(); MapStage();

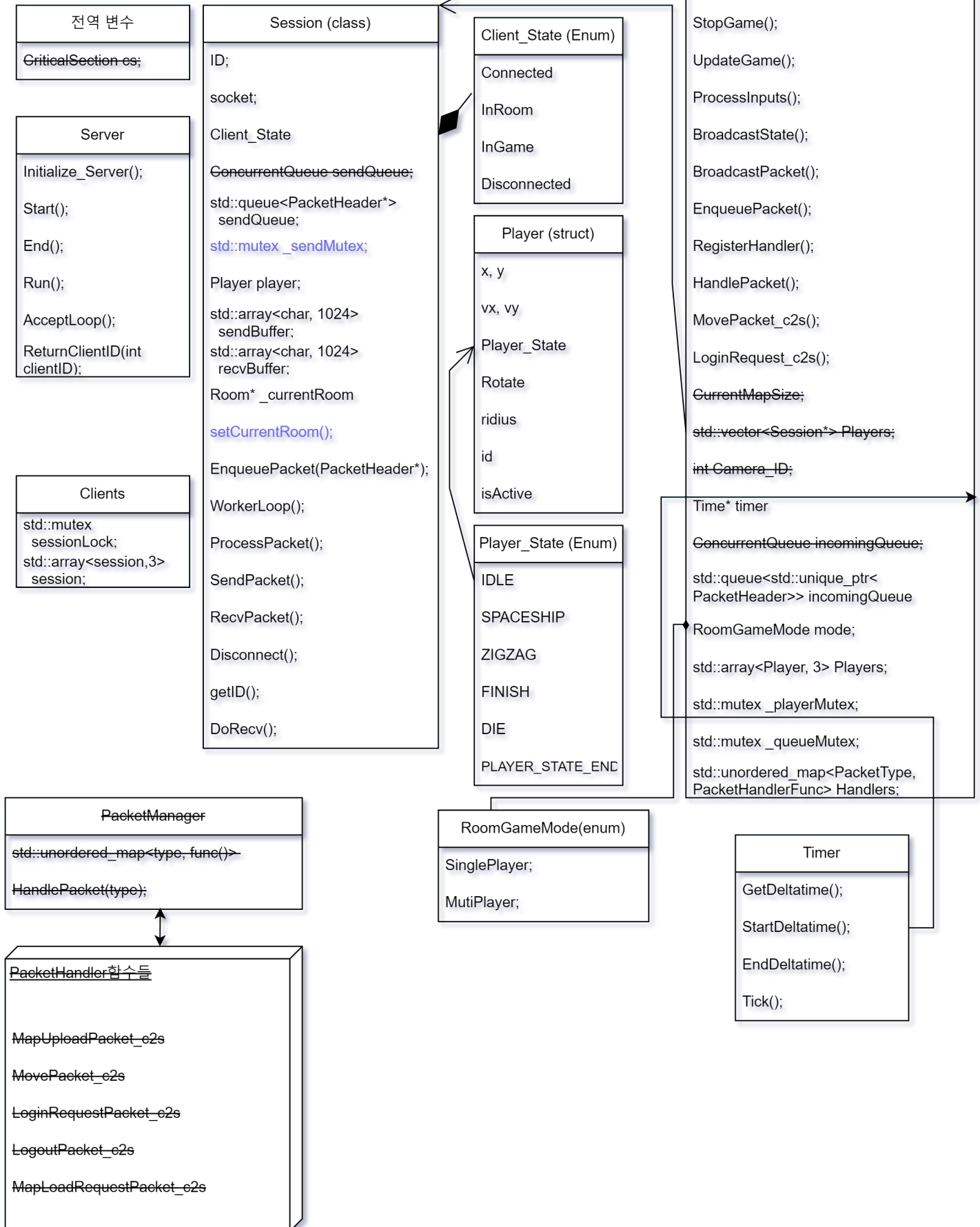
<u>MultiStage : Stage</u>
GetInstance(); DestoryInstance(); MapShow(); MapLoad(); MapStage();

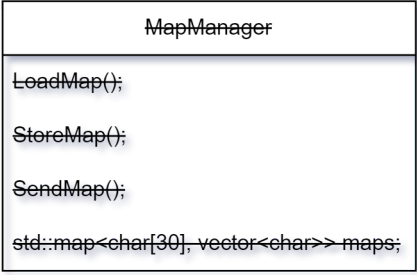
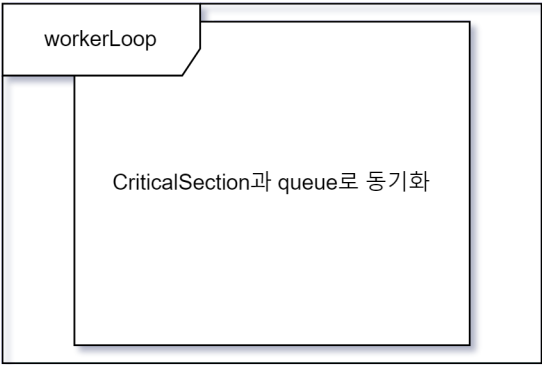
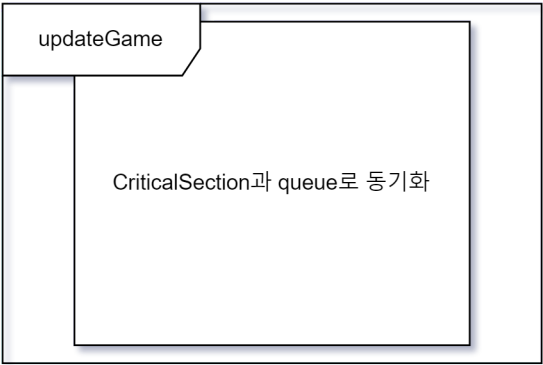
<u>UIManager</u>
GetInstance(); DestoryInstance(); MiniMapUpdate();- MiniMapShow();- ScoreBoardShow();- TimerShow();- UserNumShow();

<u>-PlayerNumUI : Object</u>
PlayerNumUI(); virtual ~PlayerNumUI(); virtual void Initailizer() override; virtual void Update(float fTime) override; virtual void LateUpdate(float fTime) override; virtual void Render(HDC mDC) override; virtual void Free() override;

<u>MiniMapUI : Object</u>
MiniMapUI();- virtual ~MiniMapUI(); virtual void Initailizer() override; virtual void Update(float fTime) override; virtual void LateUpdate(float fTime) override; virtual void Render(HDC mDC) override; virtual void Free() override;

Server Low Level Design





Packet

Client => Server

PacketManager
std::unordered_map<PacketType, func()> Handlers; RegisterHandler(type, func()); Handle(type,session);

Sever => Client

PacketManager
std::unordered_map<PacketType, func()> Handlers; RegisterHandler(type, func()); Handle(type);

MovePacket
long long size; PacketType type int id long x,y; float vx, vy; float rotate float ridius; int cameralD; Player_State player_state; float minMapPercentage;

LogoutPacket
long long size; PacketType type int id;

LoginReqeustPacket
long long size; PacketType type

MovePacket
long long size; PacketType type long x,y; float vx,vy; float rotate Player_State player_state; float minMapPercentage;

s2c_AllPlayerMovePacket
long long size; PacketType type float x[3],y[3]; float vx[3],vy[3]; float ridius[3]; Player_State state[3] float roate[3]

RoomWaitPacket
long long size; PacketType type int playerCount; int playerIDs[MAX_PLAYERS];

RoomStartPacket
long long size; PacketType type int playerCount; int playerIDs[MAX_PLAYERS];

LoginAcceptPacket
long long size; PacketType type int playerID;

SpawnOtherPlayerPacket
long long size; PacketType type int OtherplayerID;

DespawnOtherPlayerPacket
long long size; PacketType type int OtherplayerID;

PacketType(enum class)
MovePacket_s2c AllPlayerMovePacket_s2c MovePacket_c2s LoginAcceptPacket_s2c LoginRequestPacket_c2s LogoutPacket_c2s SpawnOtherPlayerPacket_s2c DespawnOtherPlayerPacket_s2c RoomWaitPacket_s2c RoomStartPacket_s2c

MapLoadRequestPacket
long long size; PacketType type char[30] mapName

MapNamesPacket
long long size; PacketType type std::array<char[30], 10> mapNames

MapLoadPacket
long long size; PacketType type char[30] mapNames vector<char> mapData;

MapUploadPacket
long long size; PacketType type char[30] mapName; vector<char> mapData;

RoomEnterAcceptPacket
long long size; PacketType type

RoomLeavePacket
long long size; PacketType type int roomId;

MapUploadDonePacket
long long size; PacketType type

RoomStartPacket
long long size; PacketType type

RoomEndPacket
long long size; PacketType type int roomId; int winUserID float ElapsedTime

역할 분담

박경준	● 서버 & 세션 (I/O) 담당, Server 클래스와 Session의 핵심 I/O(네트워크 송수신)를 담당합니다.
박대원	● Room & Game Logic 담당 Room 클래스와 게임 로직을 처리하는 UpdateGame를 담당합니다.
임찬진	● 서버의 유틸리티 & 핸들러 (Data & Packet) 담당 PacketManager, Timer, MapManager 등 유틸리티 클래스와 개별 패킷 핸들러 함수 구현을 담당합니다.

개발 환경

개발 도구	visual studio 2022
버전 관리	github
개발 언어	C++ STL
API	WINDOWS, WINSOCK2

박경준 개별 일정

		25/10/28	25/10/29	25/10/30	25/10/31	25/11/01
						Server 멤버함수 Initialize _Server(), V Start(), End() 구현 V
25/11/02	25/11/03	25/11/04	25/11/05	25/11/06	25/11/07	25/11/08
Server:: AcceptLoop() V Session 클래스 기본 구조 설계 V		workerLoop, Session::Disco nnect() V workerLoop 내 SendPacket 구현 V workerLoop 내 RecvPacket 구현 V	Room의 AddPlayer V RemovePlayer V CriticalSection V RecvBuffer 패킷 역직렬화 V, incomingQue ue.push(packe t) send(), pop() V			
25/11/09	25/11/10	25/11/11	25/11/12	25/11/13	25/11/14	25/11/15
			Room::Update Game 리팩토링, V 루프 주기 동기화, V Room의 players 스레드 동기화 V, incomingQueu e 스레드 동기화 V CriticalSection -> std::mutex 변경 V Server Room 스레드 생성 V AcceptLoop에서 room에 session 등록 V+ session의 room 등록 V			
25/11/16	25/11/17	25/11/18	25/11/19	25/11/20	25/11/21	25/11/22
		workerLoop 내 SendPacket 구현	CriticalSection sendQueue.po p()/send() NetworkMana ger::sendPack et 구현 V Client::TimeM anager init, Reset, Tick			

			,GetDeltaTime ,GetFPS 구현 V 네트워크 initialize_client호출 V			
25/11/23	25/11/24	25/11/25	25/11/26	25/11/27	25/11/28	25/11/29
		Client_State- 관리 연동	AcceptLoop- Session::workerLoop 부하- 테스트 PacketQueue 메모리 누수 해결 V 클라이언트용 스레드 생명주기 해결 V sendQueue 무한증가 해결 V 논블라킹 send,recv구현 V	윈도우 헤더와 원속2 헤더 충돌 문제 해결 V 플레이어 ID 재사용 및 관리 시스템 구축 V 클라이언트 패킷 수신 로직 개선 V LoginRequest Packet_c2s 처리부 구현 V		
25/11/30	25/12/01	25/12/02	25/12/03	25/12/04	25/12/05	25/12/06
		동시 접속/종료 예외 처리, 소켓 오류 핸들링 V	Session 인터페이스 정리/리팩토링, 서버 부하- 테스트 시연 준비 V	다른 플레이어 보간 로직 수정 V		
25/12/07	25/12/08	25/12/09	25/12/10	25/12/11	25/12/12	

박대원 개별 일정

		25/10/28	25/10/29	25/10/30	25/10/31	25/11/01
						Room 클래스 기본 구조, V Player (struct) 정의 V
25/11/02	25/11/03	25/11/04	25/11/05	25/11/06	25/11/07	25/11/08
UpdateGame 루프 V			Room멤버함수 AddPlayer, V RemovePlayer V Room::Broadc astPacket() V	Timer 클래스 구현 V , Room에 Timer 연동 V		
25/11/09	25/11/10	25/11/11	25/11/12	25/11/13	25/11/14	25/11/15
			PacketManager 연동, LoginRequest Packet_c2s- 핸들러, MapManager- 기본 구조 Room::Handle Packet, V Room::Registe rHandler 추가, V Room::MovePa cket_c2s, V Room::LoginRe questPacket_c 2s ServerPacketM anager삭제 V	MapManager 멤버함수 LoadMap(), 방 시작, V Room::LoginRe questPacket_c 2s 삭제		
25/11/16	25/11/17	25/11/18	25/11/19	25/11/20	25/11/21	25/11/22
			RoomGameMo de(enum)적용, MapUploadPa cket_c2s- 핸들러 OtherPlayer구 조 설계 V	Room::Broadc astPacket() V 핸들러: MapNamesPac ket_s2c MapLoadPack et_s2c MapLoadRequ estPacket_c2s Server::sendC lientID 구현 V		
25/11/23	25/11/24	25/11/25	25/11/26	25/11/27	25/11/28	25/11/29
			Room:: StartGame(), 핸들러: RoomStartPac ket_s2c,	Player_State- 동기화 로직, BroadcastPac ket 구현 V ,		

			SpawnOtherPl ayerPacket_s2 c V	CStageManag er::RestartSta ge()제작 (스태이지 재시작 방법 수정) V, Player::SetPla yerBitMap()제 작 V		
25/11/30	25/12/01	25/12/02	25/12/03	25/12/04	25/12/05	25/12/06
			시연 준비 V	카메라:초기화 부분 설정V, 다른 플레이어 보간 로직 수정V		
25/12/07	25/12/08	25/12/09	25/12/10	25/12/11	25/12/12	

임찬진 개별 일정

		25/10/28	25/10/29	25/10/30	25/10/31	25/11/01
25/11/02	25/11/03	25/11/04	25/11/05	25/11/06	25/11/07	25/11/08
s2c 패킷 생성, V c2s 패킷 생성 V			PacketType 생성 V		싱글톤 템플릿 구현 V TimerManager, V MapManager, V UIManager V	
25/11/09	25/11/10	25/11/11	25/11/12	25/11/13	25/11/14	25/11/15
NetworkManager V , ClientPacketManager 클래스 생성 V			Object 클래스 구현, NetworkManager::initialize_ Client() 구현 V	NetworkManager:: sendPacket(), V LoginRequest Packet_c2s 전송 V recvPacket->processPacket 으로 변경 V		
25/11/16	25/11/17	25/11/18	25/11/19	25/11/20	25/11/21	25/11/22
			LoginAcceptPacket_s2c, SingleStage, MultiStage 구현 ObjManager 모든 playerid 관리 V	NetworkManager::RecvPacket Object 클래스 구현, ObjManager 모든 playerid를 다른 방식으로 처리 V		
25/11/23	25/11/24	25/11/25	25/11/26	25/11/27	25/11/28	25/11/29
			핸들러: Handle_Room Wait_s2c V , Handle_Room Start_s2c V AllPlayer_MovePacket_c2s V RoomEnterPacket_c2s V , OtherPlayer 리소스 제작 V	per 10ms마다 위치전송 V , OtherPlayer 클래스 구현 V , OtherPlayer 3명이 id에 따라 다른 리소스로 나오도록 구현 V		
25/11/30	25/12/01	25/12/02	25/12/03	25/12/04	25/12/05	25/12/06

				UI 추가:- PlayerNumUI, MiniMapUI TimerUi, 다른 플레이어 보간 로직 수정V		
--	--	--	--	---	--	--

25/12/07	25/12/08	25/12/09	25/12/10	25/12/11	25/12/12	